



**POLITÉCNICO
ESTELLA**

**ASIGNATURA: PROGRAMACIÓN MULTIMEDIA Y
DISPOSITIVOS MÓVILES**

**GRADO SUPERIOR EN DESARROLLO DE
APLICACIONES MULTIPLATAFORMA**

PMDM TAREA 3

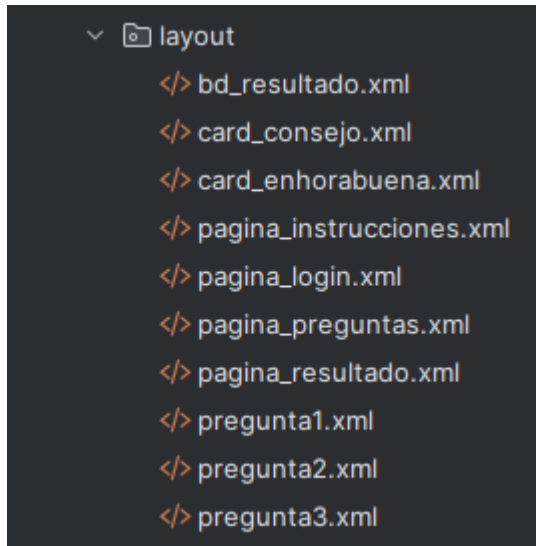
Presentado por: Eugen Moga

ÍNDICE

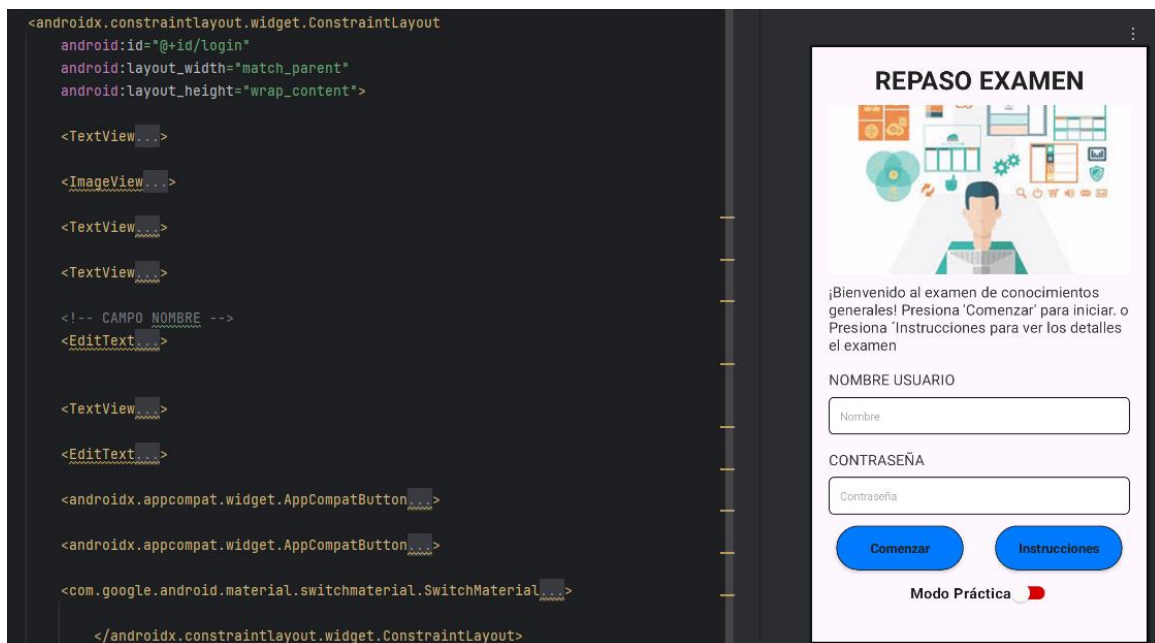
1. Diseño y maquetación (UT4).....	1
2. Navegación e interacción entre actividades (UT5)	2
3. Integración multimedia (UT6)	5
4. Persistencia de la información (UT7)	6

1. Diseño y maquetación (UT4)

Para realizar el diseño y la maquetación primero he realizado cada pantalla del diseño de figma en sus respectivos archivos xml. Asi tengo en el proyecto de Android Studio app > res > layout todas las pantallas de la aplicación.



Y cada pantalla xml tiene sus respectivos ConstraintLayout LinearLayout... con sus respectivos botones campos de texto, imágenes.



2. Navegación e interacción entre actividades (UT5)

La navegación entre las pantallas está hecha íntegramente con intents explícitos donde por ejemplo al pulsar el botón comenzar en la pantalla de login se dirige a la pantalla de preguntas

```
// LÓGICA BOTÓN COMENZAR
botonComenzar.setOnClickListener( View v -> {
    // Validar que el usuario haya ingresado un nombre
    if (validarUsuario()) {
        // Crear intent para ir a la actividad de preguntas
        Intent intent = new Intent( packageContext: MainActivity.this, PreguntasActivity.class);
        enviarDatosExtra(intent);
        startActivity(intent);
    }
});
```

EUGEN MOGA

También el botón Instrucciones va de la pantalla de login a la pantalla de instrucciones con Intent explícito

```
// LÓGICA BOTÓN INSTRUCCIONES
botonInstrucciones.setOnClickListener( View v -> {
    // Validar que el usuario haya ingresado un nombre
    if (validarUsuario()) {
        // Crear intent para ir a la actividad de instrucciones
        Intent intent = new Intent( packageContext: MainActivity.this, InstruccionesActivity.class);
        enviarDatosExtra(intent);
        startActivity(intent);
    }
});
```

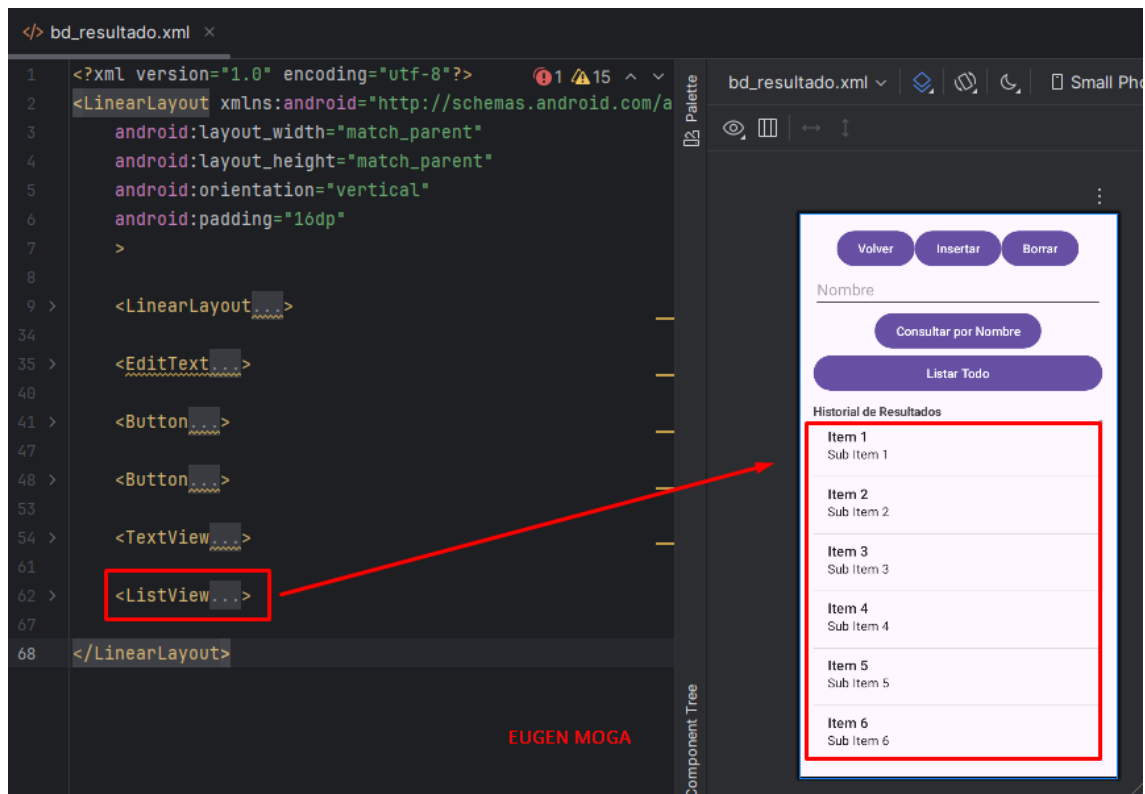
EUGEN MOGA

El intent implícito lo he utilizado en la pantalla de la pregunta tres en el botón Información para abrir un enlace externo con Intent.ACTION_VIEW

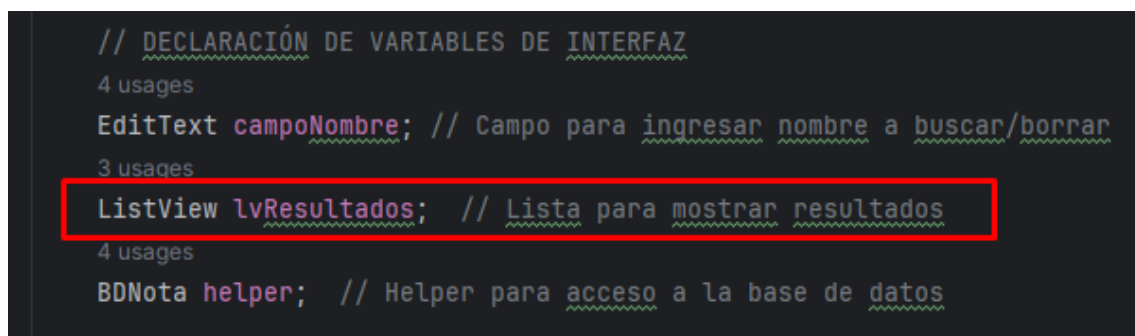
```
// BOTÓN "INFORMACIÓN" (ENLACE EXTERNO) Intent IMPLICITO
// Botón que abre información adicional en el navegador web
Button botonInformacion = findViewById(R.id.informacion);
botonInformacion.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse( urlString: "https://es.wikipedia.org/wiki/Don_Quijote_de_la_Mancha"));
        startActivity(intent);
    }
});
```

EUGEN MOGA

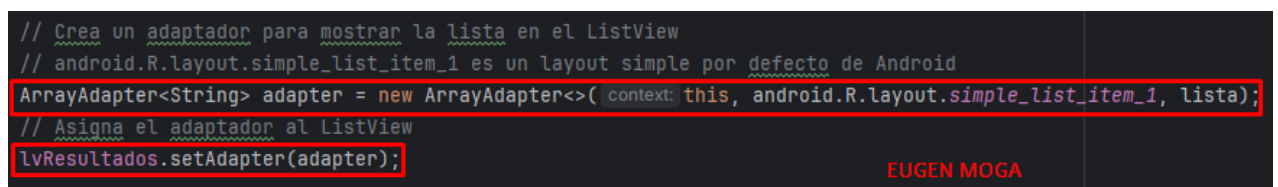
El listado con ListView lo he utilizado en bd_resultado.xml ya que la aplicación simula un test, he creado esta pantalla para guardar los resultados de cada usuario



Luego en BDActivity



En el método listarResultados creo el ArrayAdapter y le asigno los datos listados al ListView



Para gestionar los eventos de clic sobre los elementos utilizo el método setOnItemClickListener que pertenece a la clase ListView, al crear el intent agrego la información que voy a enviar a la pantalla de detalle

```

// Gestion de clic en elementos de ListView
lvResultados.setOnItemClickListener(new AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int position, long id) {

        // Obtiene el registro seleccionado en la posición 'position'
        String itemSeleccionado = (String) parent.getItemAtPosition(position);

        // Extrae el nombre del usuario del registro
        String[] lineas = itemSeleccionado.split(regex: "\\n");
        String nombreUsuario = lineas[0].replace(target: "Usuario: ", replacement: "").trim();

        // Extraer nota
        String notaCompleta = lineas[1].split(regex: "\\|")[0].replace(target: "Nota: ", replacement: "").trim();
        // Nota en formato "aciertos / totalPreguntas"
        String[] partesNota = notaCompleta.split(regex: "/");
        int aciertos = Integer.parseInt(partesNota[0]);
        int totalPreguntas = Integer.parseInt(partesNota[1]);

        // Extraer modo
        String modo = lineas[1].split(regex: "\\|")[1].replace(target: "Modo: ", replacement: "").trim();

        // Extraer fecha
        String fecha = lineas[1].split(regex: "\\|")[2].replace(target: "Fecha: ", replacement: "").trim();

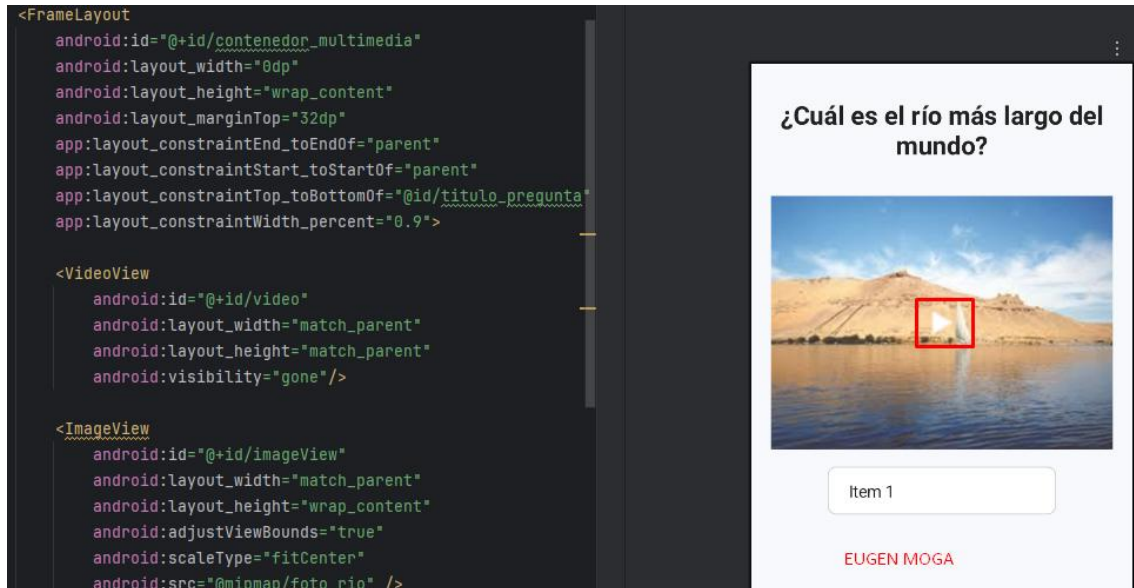
        // Crear un Intent para abrir la actividad de detalle
        Intent intent = new Intent(packageContext: BDActivity.this, DetalleActivity.class);
        intent.putExtra(name: "nombre_usuario", nombreUsuario);
        intent.putExtra(name: "aciertos", aciertos);
        intent.putExtra(name: "total_preguntas", totalPreguntas);
        intent.putExtra(name: "modo", modo);
        intent.putExtra(name: "fecha", fecha);
        startActivity(intent);
    }
});

```

EUGEN MOGA

3. Integración multimedia (UT6)

Para agregar una funcionalidad multimedia la he puesto en la pantalla de la primera pregunta el video está detrás de la imagen y solo he dejado visible el botón de play para iniciar el video, al iniciar el video el botón desaparece y la imagen también hasta que finaliza el video



Utilizo Media Controller para ofrecer controles visibles al usuario y poder controlar la reproducción en todo momento

```
// Configurar MediaController para controlar la reproducción del video
mediaController = new MediaController(context, this);
mediaController.setAnchorView(videoView);
videoView.setMediaController(mediaController);
```

```
// CONFIGURACIÓN DEL BOTÓN PARA REPRODUCIR VIDEO
botonReproducirVideo.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // Ocultar imagen estática y botón de reproducción
        imageView.setVisibility(View.GONE);
        botonReproducirVideo.setVisibility(View.GONE);

        // Mostrar video
        videoView.setVisibility(View.VISIBLE);

        // Establecer ruta del video
        String rutaVideo = "android.resource://" + getPackageName() + "/" + R.raw.video_rio;
        videoView.setVideoURI(Uri.parse(rutaVideo));

        // Listener que se ejecuta cuando el video está listo para reproducirse
        videoView.setOnPreparedListener(new MediaPlayer.OnPreparedListener() {
            @Override
            public void onPrepared(MediaPlayer mp) { videoView.start(); }
        });

        // Listener que se ejecuta cuando el video termina
        videoView.setOnCompletionListener(new MediaPlayer.OnCompletionListener() {
            @Override
            public void onCompletion(MediaPlayer mp) {
                // Restaurar estado inicial: ocultar video, mostrar imagen y botón
                videoView.setVisibility(View.GONE);
                imageView.setVisibility(View.VISIBLE);
                botonReproducirVideo.setVisibility(View.VISIBLE);
            }
        });
    }
});
```

4. Persistencia de la información (UT7)

SharedPreferences: con Shared Preferences lo utilizo para guardar el estado del switch (pantalla de inicio) entre modo Practica / Evaluación de este modo se guarda la información al utilizar algún modo en particular y al reiniciar la aplicación el modo permanece guardado.

Declaro dos constantes

```
// Constantes para almacenar/recuperar preferencias del usuario SharedPreferences
1 usage
private static final String PREFS_NAME = "MisPreferencias"; // Nombre del archivo
2 usages
private static final String MODOS_KEY = "modo_practica"; // Clave para guardar
```

Leo el valor

```
// GESTIÓN DE PREFERENCIAS (SHAREDREFERENCES)
// Obtener acceso al almacenamiento persistente de preferencias
SharedPreferences prefs = getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);

// Leer el valor guardado. Si no existe nada, ponemos 'false' (Modo Evaluación)
boolean modoPracticaActivado = prefs.getBoolean(MODO_KEY, defValue: false);
```

Configuro el switch con el valor guardado

```
// Aplicar el valor al switch y el texto correspondiente
switchModo.setChecked(modoPracticaActivado);
if (modoPracticaActivado) {
    switchModo.setText("Modo Práctica"); // Texto cuando está activado
} else {
    switchModo.setText("Modo Evaluación"); // Texto cuando está desactivado
}
```

Y guardo los cambios cuando el usuario mueve el switch

```
// Guardar cambios cuando el usuario mueve el switch
switchModo.setOnCheckedChangeListener(new CompoundButton.OnCheckedChangeListener() {
    2 usages
    @Override
    public void onCheckedChanged(CompoundButton buttonView, boolean isChecked) {
        // Actualizar texto del switch según su estado
        if (isChecked) {
            switchModo.setText("Modo Práctica");
        } else {
            switchModo.setText("Modo Evaluación");
        }

        // Guardar el nuevo estado en SharedPreferences para persistencia
        SharedPreferences.Editor editor = prefs.edit();
        editor.putBoolean(MODO_KEY, isChecked);
        editor.apply(); // Guardar cambios de forma asíncrona
    }
});
```

De este modo cuando el usuario mueve el switch a Modo Practica onCheckedChanged se ejecuta con isChecked = true y se guarda true en modo_practica

Base de datos con SQLite (Room)

Creo la clase BDEntidad y le indico que hereda de SQLiteOpenHelper. Esta clase es la que contiene la estructura de la tabla con sus respectivos datos y crea la tabla

```
// Clase helper para gestionar la base de datos SQLite de la aplicación
// Extiende SQLiteOpenHelper, que proporciona métodos para crear/actualizar la BD
4 usages
public class BDEntidad extends SQLiteOpenHelper {

    // DEFINICIÓN DE LA ESTRUCTURA DE LA TABLA
    // Sentencia SQL para crear la tabla "resultados"
    2 usages
    String crearTabla = "CREATE TABLE resultados (" +
        "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
        "nombre TEXT, " +
        "nota TEXT, " +
        "modo TEXT, " +
        "fecha TEXT)";
```

Esta clase contiene el constructor por defecto

```
// CONSTRUCTOR DE LA CLASE
// Este constructor inicializa el helper de la base de datos
// @param context: Contexto de la aplicación (necesario para acceso a BD)
// @param name: Nombre del archivo de la base de datos (ej: "resultados.db")
// @param factory: CursorFactory (puede ser null para usar el por defecto)
// @param version: Versión de la base de datos (importante para actualizaciones)
2 usages
public BDEntidad(Context context, String name, SQLiteDatabase.CursorFactory factory, int version) {
    // Llama al constructor de la clase padre (SQLiteOpenHelper)
    super(context, name, factory, version);
}
```

El método onCreate que se ejecuta cuando se crea la base de datos

```
// METODO onCreate: SE EJECUTA LA PRIMERA VEZ QUE SE CREA LA BD
// Este metodo se llama automáticamente cuando la base de datos se crea por primera vez
// @param db: Objeto SQLiteDatabase para ejecutar sentencias SQL
@Override
public void onCreate(SQLiteDatabase db) {
    // Ejecuta la sentencia SQL para crear la tabla "resultados"
    db.execSQL(crearTabla);
}
```

Y el método onUpgrade que se ejecuta cuando se actualiza la versión de la base de datos

```
// METODO onUpgrade: SE EJECUTA CUANDO SE ACTUALIZA LA VERSIÓN DE LA BD
// Este metodo se llama cuando la versión de la base de datos cambia
// Útil para migraciones de datos cuando se modifica la estructura de la BD
// @param db: Objeto SQLiteDatabase para ejecutar sentencias SQL
// @param oldVersion: Número de versión antigua de la BD
// @param newVersion: Número de versión nueva de la BD
10 usages
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    // En caso de actualizar, se borra la tabla vieja y se crea una nueva
    db.execSQL("DROP TABLE IF EXISTS resultados");
    db.execSQL(crearTabla);
}
```

En la clase ResultadoActivity tengo el método para guardar en la base de datos

```
// METODO PARA GUARDAR RESULTADOS EN BASE DE DATOS SQLite
1 usage
private void guardarResultadoEnBD(String nombreUsuario, int nota, boolean modoPractica) {
    // 1. CREAR/CONECTAR CON LA BASE DE DATOS
    BEntidad helper = new BEntidad(context: this, name: "resultados.db", factory: null, version: 1);
    SQLiteDatabase db = helper.getWritableDatabase();

    // 2. OBTENER FECHA Y HORA ACTUAL
    SimpleDateFormat sdf = new SimpleDateFormat(pattern: "dd/MM/yyyy HH:mm", Locale.getDefault());
    String fechaActual = sdf.format(new Date());

    // 3. PREPARAR DATOS PARA INSERTAR
    ContentValues valores = new ContentValues();
    valores.put("nombre", nombreUsuario);
    valores.put("nota", nota + "/" + totalPreguntas);
    valores.put("modo", modoPractica ? "Práctica" : "Evaluación");
    valores.put("fecha", fechaActual);

    // 4. INSERTAR EN LA TABLA "resultados"
    long resultado = db.insert(table: "resultados", nullColumnHack: null, valores);

    // 5. CERRAR CONEXIÓN
    db.close();
}
```

En la clase ResultadoActivity cada vez que el usuario llega a la pantalla final de la aplicación en el método onCreate ejecuto automáticamente el método guardarResultadoEnBD primero compruebo que no esté en modo revisión para no duplicar los datos.

```
// GUARDAR RESULTADO EN BASE DE DATOS
// Solo guarda en la BD si NO está en modo revisión (para evitar duplicados)
boolean modoRevisión = intentRecibido.getBooleanExtra( name: "modo_revisión", defaultValue: false);
if (!modoRevisión) {
    guardarResultadoEnBD(nombreUsuario, puntuación, modoPráctica);
}
```

En la clase DAO_BD_Activity creo los demás métodos para listar y eliminar información

Método para listar todos los resultados

```
// METODO PARA LISTAR TODOS LOS RESULTADOS
2 usages
private void listarResultados() {
    // Obtiene conexión de lectura a la BD
    SQLiteDatabase db = helper.getReadableDatabase();
    // ArrayList para almacenar los registros formateados
    ArrayList<String> lista = new ArrayList<>();

    // Ejecuta consulta SQL para obtener todos los registros
    //.rawQuery ejecuta una consulta SQL directa
    Cursor fila = db.rawQuery( sql: "SELECT * FROM resultados", selectionArgs: null);

    // Verifica si el cursor tiene resultados (moveToFirst va al primer registro)
    if (fila.moveToFirst()) {
        do {
            // Formatea cada registro como una cadena legible
            // Índices del cursor: 0=id, 1=nombre, 2=nota, 3=modo, 4=fecha
            String registro = "Usuario: " + fila.getString( columnIndex: 1) +
                "\nNota: " + fila.getString( columnIndex: 2) +
                " | Modo: " + fila.getString( columnIndex: 3)+
                " | Fecha: " + fila.getString( columnIndex: 4);
            lista.add(registro);
        } while (fila.moveToNext());
    }
    db.close();

    // Crea un adaptador para mostrar la lista en el ListView
    // android.R.layout.simple_list_item_1 es un layout simple por defecto de Android
    ArrayAdapter<String> adapter = new ArrayAdapter<>( context: this, android.R.layout.simple_list_item_1, lista);
    // Asigna el adaptador al ListView
    lvResultados.setAdapter(adapter);
}
```

Método para consultar por nombre de usuario

```
// METODO PARA CONSULTAR RESULTADOS POR NOMBRE
1 usage
private void consultarPorNombre(String nombre) {
    SQLiteDatabase db = helper.getReadableDatabase();
    ArrayList<String> listaFiltrada = new ArrayList<>();

    // Consulta SQL parametrizada para buscar por nombre
    // "SELECT * FROM resultados WHERE nombre = ?"
    // El ? será reemplazado por el valor del parámetro 'nombre'
    Cursor fila = db.rawQuery("SELECT * FROM resultados WHERE nombre = ?", new String[]{nombre});
    if (fila.moveToFirst()) {
        do {
            // Formatea cada registro encontrado
            String registro = "Usuario: " + fila.getString( columnIndex: 1) +
                "\nNota: " + fila.getString( columnIndex: 2) +
                " | Modo: " + fila.getString( columnIndex: 3) +
                " | Fecha: " + fila.getString( columnIndex: 4);
            listaFiltrada.add(registro);
        } while (fila.moveToNext());
        Toast.makeText( context: this, text: "Se han encontrado " + listaFiltrada.size() + " registros", Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText( context: this, text: "No existen registros con ese nombre", Toast.LENGTH_SHORT).show();
    }
    db.close();

    // Actualiza el ListView con los resultados filtrados
    ArrayAdapter<String> adapter = new ArrayAdapter<>( context: this, android.R.layout.simple_list_item_1, listaFiltrada);
    lvResultados.setAdapter(adapter);
}
```

Método para borrar por nombre

```
// BOTÓN "BORRAR POR NOMBRE"
botonBorrar.setOnClickListener( View v -> {
    String nombre = campoNombre.getText().toString();

    if (nombre.isEmpty()) {
        Toast.makeText( context: this, text: "Debes introducir un nombre de usuario", Toast.LENGTH_SHORT).show();
        return;
    }

    // Obtiene conexión de escritura a la BD
    SQLiteDatabase db = helper.getWritableDatabase();

    // Ejecuta la operación DELETE con parámetros para prevenir SQL injection
    // "nombre = ?" es una consulta parametrizada (segura)
    // new String[]{nombre} reemplaza el ? por el valor del nombre
    int cantidad = db.delete( table: "resultados", whereClause: "nombre = ?", new String[]{nombre});
    db.close();
    campoNombre.setText("");
    if (cantidad > 0) {
        Toast.makeText( context: this, text: "Borrado con éxito", Toast.LENGTH_SHORT).show();
        listarResultados();
    } else {
        Toast.makeText( context: this, text: "No se ha encontrado el usuario", Toast.LENGTH_SHORT).show();
    }
});
```

En el video enseñé como funcionan todos los metodos